Lattice Signature with Efficient Protocols

Bataille Alex, Clerbaux Coline
Fallik Gabriel, Grasso Francesca, Tajani Mohamed

March 27, 2026

1 Introduction

Digital signatures are fundamental cryptographic primitives, allowing a user to authenticate a message and guarantee its integrity. For decades, most practical signature schemes have relied on classical hardness assumptions such as integer factorization or discrete logarithms. However, these assumptions are known to be broken by quantum algorithms, which motivates the search for alternative constructions secure in the post-quantum setting. Lattice-based cryptography has emerged as one of the most promising frameworks for post-quantum security, as its underlying problems are believed to remain hard even for quantum computers. Beyond security, modern applications often require signatures to interact efficiently with zero-knowledge proof systems, for instance in privacy-preserving protocols. Designing lattice-based signatures that remain compatible with such proof systems is therefore crucial. A first attempt in this direction was the construction of Libert, Ling, Mouhartem et al. ([**LLM+16**]), which proposes a lattice-based signature scheme with efficient protocols. Although conceptually appealing, their construction suffers from extremely large proof sizes (several hundreds of megabytes) making it impractical. The paper studied in this report ([**mainPaper**]) revisits this problem and proposes a new lattice-based signature scheme specifically designed to interface smoothly with recent zero-knowledge argument systems, while reducing the size of the produced proofs. In the following, we analyse the main ideas behind this construction, highlight its improvements over previous work, and examine some of its technical components in more detail.

2 Paper Contributions

2.1 Large Sizes Problem

As mentioned in the introduction, the signature scheme from Libert et al. [**LLM+16**] suffers from large sized proofs (30 to 40 times larger). In this section, we study why this previous scheme yields such enormous sizes. In [**LLM+16**], the signature scheme relies on a commitment $\mathbf{c} = \mathbf{D_0 r} + \mathbf{D_1 m}$. To prove knowledge of a valid signature on a hidden message, the prover must demonstrate knowledge of the opening $(\mathbf{r}, \mathbf{m})$ such that $\mathbf{c}$ is well formed. More precisely, the verification algorithm operates on the binary decomposition of the commitment vector. In our case, $\mathbf{c} \in \mathbb{Z}_q^n$ so the protocol requires decomposing

each coefficient c_i into $l = \lceil \log_2 q \rceil$ bits.
This design choice induces 2 main inefficiencies :

1. Witness expansion : The witness size for the ZKAoK scales linearly with the number of bits to be proven. Instead of proving knowledge of n elements in Z_q^n, the prover must prove knowledge of $n \cdot \log_2 q$ binary variables.

2. Constraint proliferation : To ensure the decomposition is valid, the ZK protocol must enforce the binary constraint for every bit. In frameworks based on quadratic constraints, this requires adding $\mathcal{O}(n \log q)$ quadratic equations to the proof system. Furthermore, the security reduction in [**LLM+16**] relies on embedding a trapdoor within the decomposition matrix. This introduces a norm growth factor $\propto \sqrt{n \log q}$ in the extracted solution, forcing the scheme to adopt overly conservative parameters to maintain security margins.

2.2 The Solution

To mitigate this, we optimize the interactions between the commitment scheme implicitly used by privacy-enhancing protocols and the signature scheme itself. Instead of generating the public parameters of these 2 components independently from one another, the authors propose a unified generation, using a common matrix $\mathbf{A}$. This indeed reduces the size of the proofs since the size of the public key is smaller and it allows the commitment randomness $\mathbf{r}$ to be algebraically absorbed into the signature vector $\mathbf{v}$, eliminating the need for a separate proof of opening.

3 Signature Scheme

The signature scheme proposed by the paper is based on the one studied in [**LLM+16**], along with the improvements discussed in the previous section.

The proposed construction is a lattice-based signature scheme with efficient protocols, designed to minimize the witness size required for subsequent zero-knowledge proofs. While it inherits the high-level goal of [**LLM+16**] (signing committed messages), its internal mechanics differ fundamentally to achieve the efficiency gains outlined in Section 2.2. The scheme relies on the hardness of the Short Integer Solution (SIS) problem over lattices and utilizes the trapdoor sampling framework of Micciancio and Peikert [**MP12**].

3.1 Key Generation and Parameters

The scheme operates over a modulus q and dimensions n, m. The key generation process (Algorithm 3.2 in [**mainPaper**]) produces a public key $\mathsf{pk} = (\mathbf{A}, \mathbf{B}, \mathbf{u})$ and a secret key $\mathsf{sk} = \mathbf{R}$.

- $\mathbf{A} \in \mathbb{Z}_q^{n \times m_1}$ is chosen uniformly at random. Crucially, in the context of signing hidden messages, $\mathbf{A}$ is derived from a public seed via a hash function (modeled as a Random Oracle) to ensure it is statistically random and independent of the trapdoor.

- $\mathbf{R} \in \{-1, 0, 1\}^{m_1 \times m_2}$ is a short random matrix serving as the *trapdoor*.

- $\mathbf{B} = \mathbf{AR} \pmod q$ is computed such that the relation between $\mathbf{A}$ and $\mathbf{B}$ is hidden by the shortness of $\mathbf{R}$.
- $\mathbf{u} \in \mathbb{Z}_q^n$ is a random syndrome defining the specific lattice coset for signatures.

The public parameters also include a gadget matrix $\mathbf{G}$ and a commitment key $\mathbf{D}$, which are used during the signing and verification phases.

3.2 Signing Committed Messages

The core innovation lies in the signing algorithm (Algorithm 3.3). To sign a binary message $\mathbf{m}$ hidden inside a commitment $\mathbf{c}$, the protocol proceeds as follows:

1. **Commitment:** The user computes a commitment to $\mathbf{m}$ using the public matrix $\mathbf{A}$:
$$\mathbf{c} = \mathbf{Ar} + \mathbf{Dm} \pmod q \tag{1}$$
where $\mathbf{r}$ is a Gaussian randomness vector. Note that $\mathbf{A}$ is shared between the signature key and the commitment scheme.

2. **Preimage Sampling:** The signer chooses a tag τ and uses the trapdoor $\mathbf{R}$ to sample a short vector $\mathbf{v}'$ satisfying a modified linear equation:
$$[\mathbf{A} \mid \tau\mathbf{G} - \mathbf{B}]\mathbf{v}' = \mathbf{u} + \mathbf{c} \pmod q \tag{2}$$
Substituting $\mathbf{c} = \mathbf{Ar} + \mathbf{Dm}$ and $\mathbf{B} = \mathbf{AR}$, the equation becomes:
$$\mathbf{Av}_1' + (\tau\mathbf{G} - \mathbf{AR})\mathbf{v}_2' = \mathbf{u} + \mathbf{Ar} + \mathbf{Dm} \tag{3}$$

3. **Randomness Merging:** The final signature vector $\mathbf{v}$ is constructed by subtracting the commitment randomness $\mathbf{r}$ from the first part of $\mathbf{v}'$:
$$\mathbf{v} = \mathbf{v}' - \begin{bmatrix} \mathbf{r} \\ \mathbf{0} \end{bmatrix} \tag{4}$$
The resulting signature is the pair $\sigma = (\tau, \mathbf{v})$.

3.3 Verification

Verification (Algorithm 3.4) checks the validity of the signature without knowing $\mathbf{r}$. The verifier computes:

$$[\mathbf{A} \mid \tau\mathbf{G} - \mathbf{B}]\mathbf{v} \stackrel{?}{=} \mathbf{u} + \mathbf{Dm} \pmod q \tag{5}$$

By substituting $\mathbf{v} = \mathbf{v}' - [\mathbf{r}^T \mid \mathbf{0}^T]^T$, the term $\mathbf{Ar}$ from the commitment cancels out with the $\mathbf{Ar}$ implicitly contained in $\mathbf{v}'$. This algebraic cancellation is the key mechanism: it allows the verifier to check the signature against the message $\mathbf{m}$ directly, without needing to verify the commitment opening $\mathbf{r}$ separately. Additionally, the verifier checks that $\|\mathbf{v}\|$ is sufficiently small to ensure security against lattice reduction attacks.

3.4 Security Considerations

The reuse of matrix $\mathbf{A}$ for both the signature trapdoor and the commitment introduces a potential vulnerability: if the signer could choose $\mathbf{A}$ maliciously, they might break the hiding property of the commitment. To prevent this, the scheme requires $\mathbf{A}$ to be generated via a hash function (Random Oracle Model) or a trusted setup, ensuring $\mathbf{A}$ is statistically uniform and independent of the trapdoor $\mathbf{R}$. The security of the scheme is proven under the SIS assumption, distinguishing between Type I forgeries (new tag) and Type II forgeries (reused tag), with the latter requiring the use of Rényi divergence to handle the noise distribution introduced by the randomness merging.

4 Application to E-Voting

4.1 System Architecture

We map the anonymous credentials construction of Section 5.3 of [**JRS23**] to an e-voting system. The mapping is shown in Figure **??** and Table **??**.

Table 1: Mapping from Anonymous Credentials to E-Voting

Paper (Section 5.3)	**E-Voting System**	**Algorithm**
Organization	Election Authority	—
OKeyGen	Election key generation	Alg. 5.3 $\rightarrow$ Alg. 3.2
UKeyGen	Voter registration	Alg. 5.4
Issue	Credential issuance	Alg. 5.5 (oblivious signing)
Show	Vote casting	Alg. 5.6 (ZK proof)
Tag $\tau = F(\mathsf{st})$	Anti-double-vote serial	Stateful, injective F
Bulletin board	Public vote record	Tags + proofs

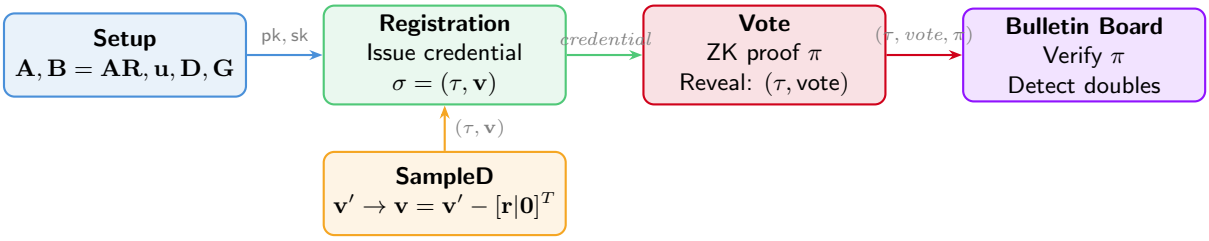

Figure 1: E-voting pipeline based on the lattice SEP construction.

4.2 Protocol Description

The e-voting protocol proceeds in four phases.

Phase 1: Election Setup. The election authority runs KeyGen (Algorithm 3.2) to generate its signing keys $(\mathsf{pk}, \mathsf{sk})$. The public parameters pp and the public key $\mathsf{pk} = (\mathbf{A}, \mathbf{B}, \mathbf{u})$ are published. The secret key $\mathsf{sk} = \mathbf{R}$ is stored securely.

Phase 2: Voter Registration. Each voter generates a secret key $\mathbf{s} \leftarrow_{\$} \{0,1\}^{m_s}$ via $\mathsf{UKeyGen}$ (Algorithm 5.4). The voter then interacts with the authority through the Issue protocol (Algorithm 5.5): the authority signs the message $\mathbf{m}_e = [\mathbf{s} \| \text{attributes}]$ via oblivious signing, producing a credential $\sigma = (\tau, \mathbf{v})$. The tag $\tau = F(\mathsf{st})$ is unique per voter, where F is an injective function and st is a counter incremented after each issuance.

Phase 3: Vote Casting. To cast a vote, the voter runs the Show protocol (Algorithm 5.6): they produce a zero-knowledge proof π demonstrating that they possess a valid credential $(\tau, \mathbf{v})$ on some message $\mathbf{m}_e$ satisfying $\mathsf{Verify}(\mathsf{pk}, \mathbf{m}_e, (\tau, \mathbf{v})) = 1$. The proof reveals *only* the tag τ and the vote — the voter's identity, secret key, and other attributes remain hidden.

Phase 4: Verification and Tallying. The bulletin board accepts a ballot (τ, vote, π) if and only if (i) the proof π verifies, and (ii) the tag τ has not been used before. If a voter attempts to vote twice, their tag will already appear on the board and the second ballot is rejected.

4.3 Security Properties

The system achieves the following security properties, directly inherited from the anonymous credentials construction:

1. **Anonymity.** The bulletin board reveals only (τ, vote). The link between a voter's identity and their tag is protected by the zero-knowledge property of the proof system (LNP22 framework [**LNP22**]).

2. **Unforgeability.** Creating a valid credential without the authority's secret key requires solving the SIS problem, which is believed to be hard even for quantum computers. The EUF-CMA security of the signature reduces to $\mathrm{SIS}^{\infty,2}_{n,m_1,q,\beta}$ via the proofs in Lemmas 3.2 and 3.3 of [**JRS23**].

3. **Double-vote prevention.** Each voter receives a unique tag τ. Since tags are revealed on the bulletin board, any duplicate is immediately detected.

4. **Post-quantum security.** The underlying Module-SIS and Module-LWE problems are not known to be solvable in polynomial time by quantum algorithms.

5 Implementation

5.1 Code Structure

Our implementation is organized into five Python modules, with a clear separation between the lattice primitives and the e-voting application layer:

Table 2: Source code organization

Module	Description
`lattice_sampler.py`	`SampleD` algorithm with corrected gadget decomposition, `gadget_solve` base-2 decomposition
`signature.py`	`LatticeSignatureScheme` class implementing Algorithms 3.1–3.4
`evoting.py`	`ElectionAuthority`, `BulletinBoard`, `Voter` classes mapping Section 5.3
`parameters.py`	`ParameterAnalyzer` reproducing Table H.4
`main.py`	Entry point running the full simulation

The `truncated-sampler` repository by Jeudy and Sanders [**JS25**] is included as a Git submodule. It provides (i) Python scripts for parameter estimation, which we used to validate our reproduction of Table H.4, and (ii) a C implementation of the full signature scheme with FLINT-based polynomial arithmetic, which can be compiled separately for performance benchmarks.

5.2 Key Implementation Details

Gadget matrix. We construct $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g}$ using NumPy's `np.kron`, producing a matrix of size $n \times nk$ where $k = \lceil \log_2 q \rceil = 12$ for $q = 3329$.

Gadget decomposition. The `gadget_solve` function decomposes an integer $t \bmod q$ into its base-2 representation, returning a vector $\mathbf{z} \in \{0,1\}^k$ such that $\mathbf{g}^T\mathbf{z} = t \bmod q$.

Tag inversion. The modular inverse $\tau^{-1} \bmod q$ is computed via Fermat's little theorem as $\tau^{q-2} \bmod q$, leveraging Python's built-in modular exponentiation.

Randomness absorption. The signature absorbs the commitment randomness by computing $\mathbf{v} = \mathbf{v}' - [\mathbf{r}^T \mid \mathbf{0}]^T$, which algebraically cancels the $\mathbf{Ar}$ term during verification, as shown in Equation (**??**).

5.3 Toy vs. Real Parameters

Our implementation uses toy parameters ($n = 64$, $q = 3329$, $m_1 = 128$) for computational feasibility. The real parameters from Table H.4 of [**JRS23**] are shown in Table **??**.

6 Parameter Analysis

6.1 Reproduction of Table H.4

Table **??** presents our reproduction of the parameter estimates from Appendix H of [**JRS23**] for the anonymous credentials construction with 10 attributes and 4 disclosed.

Table 3: Reproduced parameters from Table H.4 (128-bit quantum security)

Parameter	**Symbol**	**Value**
Ring degree	n	128
Module rank	d	12
Modulus	q	$2^{44} - 119$
Splitting factors	k	4
Tag weight	w	6
Tag space size	$\lvert T_w \rvert$	$\approx 2^{32.3}$
Commitment randomness dim.	m_1	657
Trapdoor rank	m_2	528
User secret dim.	m_s	24
Attribute polynomials	m_3	10
Preimage sampling width	σ	5 609
Combined width	σ_1	26 587
Commitment randomness width	σ_2	25 989

6.2 Comparison with Prior Work

Table **??** reproduces Table 1.1 from [**JRS23**], comparing the signature scheme with the only prior lattice-based SEP construction [**LLM16**].

Table 4: Efficiency comparison (128-bit quantum security, $\lambda = 128$)

Metric	**LLM$^+$16 [LLM16]**	**This paper (std.)**	**This paper (module)**
$\lvert \mathsf{pk} \rvert$	707×10^4 MB	296×10 MB	**7.8 MB**
$\lvert \mathsf{sk} \rvert$	372×10^2 MB	229×10 MB	**8.9 MB**
$\lvert \sigma \rvert$	2139×10^2 KB	418 KB	**273 KB**
$\lvert \pi \rvert$ (ZK proof)	2671×10^3 KB	177×10^2 KB	**639 KB**

The module instantiation achieves improvements of approximately 900× for the public key, 4 000× for the secret key, 780× for the signature, and 4 000× for the ZK proof, compared to [**LLM16**].

6.3 E-Voting Scalability

We evaluate the practical feasibility of deploying the scheme for elections of various sizes. The dominant cost is the storage of ZK proofs on the bulletin board, at 724 KB per voter.

Table 5: Bulletin board storage requirements

Number of voters	**Total storage**
1 000	0.7 GB
10 000	6.9 GB
100 000	69 GB
1 000 000	691 GB

For comparison, the pairing-based Helios system [**Helios09**] produces proofs of approximately 2–5 KB, yielding an overhead factor of roughly 240×. However, Helios is not post-quantum secure.

The tag space $T_w = \{\tau \in S_{\text{bin}} : \|\tau\|_2 = \sqrt{w}\}$ with $w = 6$ and $n = 128$ has cardinality $\binom{128}{6} \approx 5.4 \times 10^9$, sufficient for any national election.

7 Experimental Results

We ran our simulation with 10 voters and 3 candidates using toy parameters ($n = 64$, $q = 3329$). The results confirm the correctness and security properties of the implementation.

Table 6: Simulation results (toy parameters)

Metric	**Result**
Signatures issued	10/10 valid
Votes cast and verified	10/10 accepted
Double-vote attempts	1 detected and rejected
Forgery attempts	1 correctly rejected
Signing time (avg.)	< 1 ms
Verification time (avg.)	≈ 2 ms
Signature size (toy)	7.0 KB
Public key size (toy)	448.5 KB

All 10 signatures passed the verification equation $\mathbf{A}_\tau \mathbf{v} = \mathbf{u} + \mathbf{D}\mathbf{m} \bmod q$ and the norm bounds $\|\mathbf{v}_1\|_\infty \leq \sigma_1 \log_2 m_1$ and $\|\mathbf{v}_2\|_\infty \leq 1$ (since $\mathbf{v}_2$ contains binary decomposition digits). The double-vote attempt was rejected because the tag $\tau = 1$ had already been recorded on the bulletin board. The forgery attempt was rejected because a random vector $\mathbf{v}$ has negligible probability of satisfying the verification equation.

8 Conclusion